Portfolio Project Documentation

# Rag Doc Qa

Deep-dive technical documentation — 15 sections

Generated April 07, 2026

Introduction

# Readme

# Deep Documentation — RAG Document Q&A

This documentation provides a **line-by-line explanation** of every file in the project, the concepts behind each decision, and interview-ready talking points.

## Table of Contents

Section 1

# Architecture

# Architecture Deep Dive

## The Big Picture

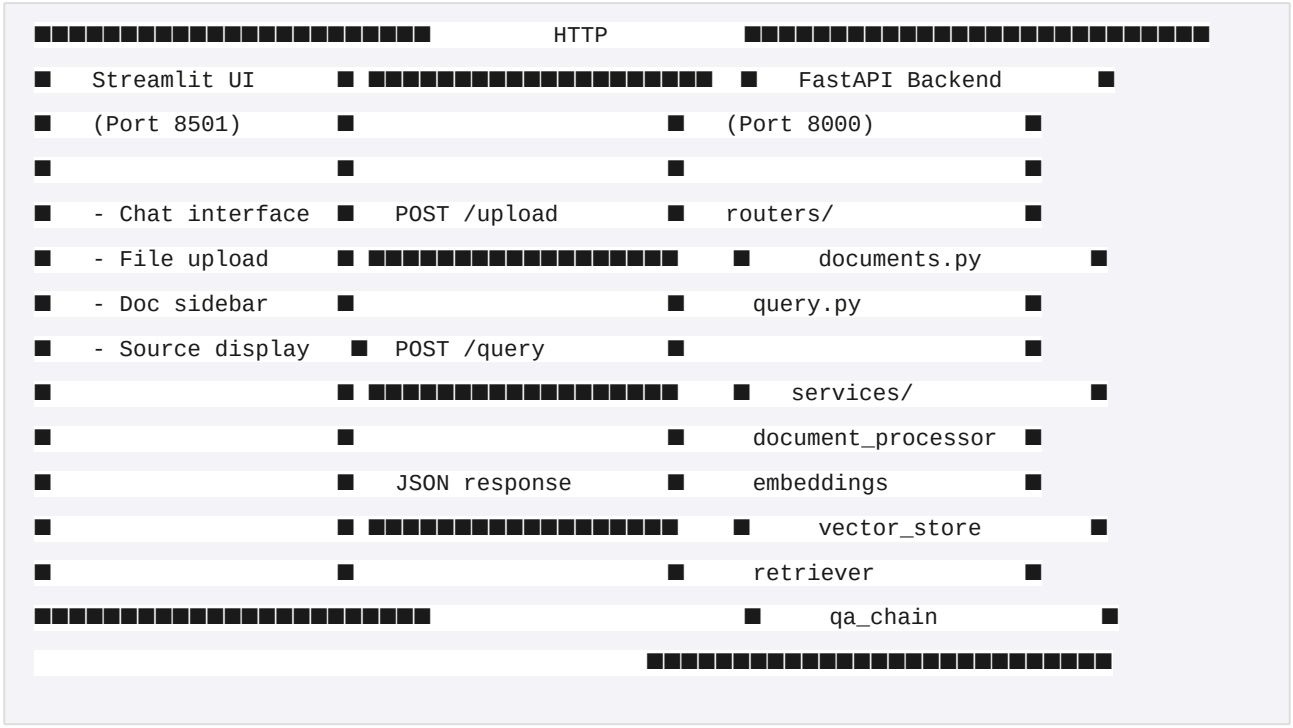
This project is a **RAG (Retrieval-Augmented Generation)** system. Instead of asking an LLM to answer from its training data (which can hallucinate), we:

1. **Store** your documents as searchable chunks
2. **Retrieve** the most relevant chunks for a given question
3. **Generate** an answer using only those chunks as context

This is the most common pattern in production AI applications today.

## Two Entry Points

The system has two independent processes that communicate over HTTP:




**Why separate them?** This is how production apps work. The frontend and backend can be deployed independently, scaled independently, and developed independently. The Streamlit UI could be replaced with a React app without touching the backend.

## Document Upload Flow (What happens when you upload a file)

```
Step 1: User uploads PDF/TXT/DOCX via Streamlit
        ■
        ▼
Step 2: Streamlit sends file to FastAPI  →  POST /documents/upload
        ■
        ▼
Step 3: document_processor.py receives raw file bytes
        ■
        ■■■ Detects file type (.pdf, .txt, .docx)
        ■■■ Writes to a temporary file (loaders need file paths)
        ■■■ Loads with appropriate LangChain loader:
        ■     PyPDFLoader   → extracts text per page
        ■     TextLoader    → reads as plain text
        ■     Docx2txtLoader → extracts text from Word docs
        ■
        ▼
Step 4: Text Chunking
        ■
        ■■■ RecursiveCharacterTextSplitter breaks text into ~1000 char pieces
        ■■■ Each chunk overlaps the previous by ~200 chars
        ■■■ WHY? An LLM can't process a 50-page PDF at once. Smaller chunks
        ■   allow precise retrieval — you find the exact paragraph that answers
        ■   the question, not the entire document.
        ■
        ▼
Step 5: Embedding
        ■
        ■■■ sentence-transformers model (all-MiniLM-L6-v2) converts each
        ■   chunk into a 384-dimensional vector (list of 384 numbers)
        ■■■ These vectors capture the MEANING of the text, not just keywords
        ■■■ Similar meanings → vectors that are close together in 384D space
        ■
        ▼
Step 6: Storage in ChromaDB
        ■
        ■■■ Each chunk is stored with:
        ■     - The vector (for similarity search)
        ■     - The original text (to show to the LLM)
        ■     - Metadata: {doc_id, filename, page_number, chunk_index}
        ■■■ ChromaDB persists to disk at ./chroma_db/
```

```
        ■
        ▼
Step 7: Response sent back with summary (doc_id, num_chunks, etc.)
```

## Query Flow (What happens when you ask a question)

```
Step 1: User types question in Streamlit chat
        ■
        ▼
Step 2: Streamlit sends question to FastAPI  →  POST /query
        ■
        ▼
Step 3: retriever.py runs HYBRID SEARCH (two searches in parallel)
        ■
        ■■■ SEMANTIC SEARCH (ChromaDB):
        ■     1. Embed the question into a 384D vector
        ■     2. Find chunks whose vectors are closest (cosine similarity)
        ■     3. Good at: "What are the findings?" matching "The results show..."
        ■     4. Bad at: exact names like "ISM001-055"
        ■
        ■■■ KEYWORD SEARCH (BM25):
        ■     1. Tokenize the question into words
        ■     2. Score chunks by term frequency / inverse document frequency
        ■     3. Good at: exact matches like "Recursion Pharmaceuticals"
        ■     4. Bad at: paraphrased questions
        ■
        ■■■ RECIPROCAL RANK FUSION (RRF):
        ■     1. Both searches return ranked lists
        ■     2. RRF combines them: score = weight / (60 + rank)
        ■     3. Semantic gets 70% weight, BM25 gets 30%
        ■     4. Chunks that appear in BOTH lists get boosted
        ■     5. Take top 5 chunks
        ■
        ▼
Step 4: qa_chain.py builds a prompt for Claude
        ■
        ■■■ Formats the 5 retrieved chunks with source labels
        ■■■ Adds the prompt template:
        ■     "Answer ONLY from the context. Cite sources as [Source: file, Page X]"
```

```
      ■■■ Includes chat history for follow-up questions
       ■
       ▼
Step 5: Claude generates an answer with citations
       ■
       ▼
Step 6: Response sent back: {answer, sources[], time_taken}
       ■
       ▼
Step 7: Streamlit displays the answer with expandable source references
```

## Why This Architecture Matters

| Design Choice | Why |
|---|---|
| Separate frontend/backend | Production pattern, independently deployable |
| Lazy-loaded models | Embedding model only loads on first use, not at import time |
| Hybrid search | Covers both semantic understanding AND exact keyword matching |
| Metadata on every chunk | Enables citations, filtering, and document management |
| ChromaDB with cosine similarity | Industry standard for RAG vector search |
| Chunk overlap | Prevents losing context at chunk boundaries |

Section 2

# Config

# config.py — Centralized Settings

**File:** `app/config.py`

## What This File Does

This is the **single source of truth** for all configurable values in the project. Instead of scattering `os.getenv()` calls across every file, we define all settings in one place using Pydantic's `BaseSettings`.

## Line-by-Line Explanation

```
from pydantic_settings import BaseSettings
```

**Line 1:** Imports `BaseSettings` from the `pydantic-settings` library. This is a special class that automatically reads values from environment variables and `.env` files. It also validates types — if you accidentally set `chunk_size="abc"`, it will throw an error at startup rather than failing silently later.

```
class Settings(BaseSettings):
```

**Line 4:** Defines our settings class. By inheriting from `BaseSettings`, every field becomes an environment variable. For example, `chunk_size` can be overridden by setting `CHUNK_SIZE=500` in your `.env` file.

```
    anthropic_api_key: str = ""
```

**Line 5:** The API key for Claude. Defaults to empty string so the app can start without it (needed for tests that don't call Claude). The actual validation happens in `qa_chain.py` when the key is first used.

**Why not make it required?** Originally it was `anthropic_api_key: str` (required). But this broke unit tests for embeddings and chunking — tests that never call Claude would crash because Pydantic couldn't find the API key. By defaulting to `""` and validating at point-of-use, we keep tests fast and isolated.

```
    # Chunking
    chunk_size: int = 1000
    chunk_overlap: int = 200
```

**Lines 8-9:** Controls how documents are split into pieces.
- `chunk_size=1000`: Each chunk is roughly 1000 characters (~150-200 words). This is the industry-standard sweet spot. Too small (200) = chunks lack context. Too large (5000) = retrieval

becomes imprecise.
- `chunk_overlap=200`: Adjacent chunks share 200 characters. This prevents losing information that spans a chunk boundary. If a sentence is split between chunk 3 and chunk 4, the overlap ensures both chunks contain the full sentence.

```
    # Embedding
    embedding_model: str = "all-MiniLM-L6-v2"
```

**Line 12:** The sentence-transformers model used to convert text into vectors.
- `all-MiniLM-L6-v2` produces 384-dimensional vectors
- It's the most popular lightweight embedding model (90M+ downloads)
- Runs locally, no API call needed
- Good balance of speed and quality for a portfolio project

```
    # Vector store
    chroma_persist_dir: str = "./chroma_db"
    chroma_collection_name: str = "documents"
```

**Lines 15-16:** Where ChromaDB stores its data on disk, and the name of the collection (like a "table" in a database). Persisting to disk means your uploaded documents survive server restarts.

```
    # Retrieval
    max_retrieval_k: int = 5
    semantic_weight: float = 0.7
    keyword_weight: float = 0.3
```

**Lines 19-21:** Controls the hybrid search behavior.
- `max_retrieval_k=5`: Return the top 5 most relevant chunks. More chunks = more context for Claude but higher cost and potential "lost in the middle" issues.
- `semantic_weight=0.7`: Semantic (meaning-based) search gets 70% influence.
- `keyword_weight=0.3`: BM25 (keyword-based) search gets 30% influence.
- **Why 70/30?** Semantic search is generally more useful (understands paraphrasing), but keyword search catches exact terms that semantic misses. The 70/30 ratio is a well-tested default in information retrieval research.

```
    # LLM
    llm_model: str = "claude-sonnet-4-20250514"
    llm_temperature: float = 0
```

**Lines 24-25:**
- `llm_model`: Which Claude model to use for answer generation.
- `llm_temperature=0`: Makes Claude's output deterministic — the same question with the same context always gives the same answer. For a Q&A system, you want consistency, not creativity.

```
    # File storage
    upload_dir: str = "./data"
```

**Line 28:** Directory for uploaded files (currently unused — we process files in memory via temp files — but available for future features like file previews).

```
    model_config = {"env_file": ".env"}
```

**Line 30:** Tells Pydantic to look for a `.env` file in the current directory. Any variable in `.env` (like `ANTHROPIC_API_KEY=sk-...`) automatically populates the corresponding field.

```
settings = Settings()
```

**Line 33:** Creates a single global instance. Every other file imports this one instance: `from app.config import settings`. This is the **Singleton pattern** — one settings object shared across the entire app.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Pydantic Settings** | Type-safe configuration management that reads from env vars and `.env` files |
| **Singleton Pattern** | One shared instance (`settings`) used everywhere |
| **Environment Variables** | External configuration that keeps secrets out of code |
| **Configuration as Code** | All tuneable parameters in one place, with sensible defaults |

## Why This Matters for Interviews

"I centralized all configuration using Pydantic Settings, which gives me type validation, environment variable binding, and `.env` file support in one class. This means if someone deploys the app with

`CHUNK_SIZE=abc`, it fails fast at startup with a clear error instead of crashing at runtime."

Section 3

# Models

# models.py — Request/Response Schemas

**File:** `app/models.py`

## What This File Does

Defines the **shape of every request and response** in the API using Pydantic models. FastAPI uses these to:
1. **Validate** incoming requests (reject bad data with clear error messages)
2. **Serialize** responses (convert Python objects to JSON)
3. **Generate API docs** (the Swagger UI at `/docs` reads these schemas)

## Line-by-Line Explanation

```
from pydantic import BaseModel
```

**Line 1:** `BaseModel` is the foundation for all Pydantic data models. Any class that extends it gets automatic validation, serialization, and documentation.

---

### UploadResponse

```
class UploadResponse(BaseModel):
    doc_id: str
    filename: str
    num_chunks: int
    num_pages: int | None
    message: str
```

**Lines 4-9:** What the API returns after a successful file upload.
- `doc_id: str` — A UUID that uniquely identifies this document. Used for deletion and chunk inspection.
- `filename: str` — The original filename the user uploaded.
- `num_chunks: int` — How many pieces the document was split into. Gives the user a sense of document size.
- `num_pages: int | None` — Page count for PDFs. `None` for TXT/DOCX (they don't have "pages"). The `int | None` syntax (Python 3.10+) means "this can be an integer OR null in JSON."
- `message: str` — Human-readable summary like "Successfully processed 'report.pdf' into 12 chunks."

---

### DocumentInfo

```
class DocumentInfo(BaseModel):
    doc_id: str
    filename: str
    num_chunks: int
    num_pages: int | None
```

**Lines 12-16:** A lighter version of UploadResponse used when listing all documents. Same fields minus the `message` — when listing 10 documents, you don't need 10 success messages.

---

### SourceReference

```
class SourceReference(BaseModel):
    filename: str
    page: int | None
    chunk_index: int
    snippet: str
```

**Lines 19-23:** Represents one source citation in an answer. This is what makes our RAG system trustworthy — users can verify the answer against the original text.
- `filename` — Which document this chunk came from.
- `page` — Which page (1-indexed, human-friendly). `None` for non-PDF files.
- `chunk_index` — Which chunk within the document (0-indexed).
- `snippet` — First 200 characters of the chunk, so the user gets a preview without seeing the full text.

---

### QueryRequest

```
class QueryRequest(BaseModel):
    question: str
    chat_history: list[dict[str, str]] = []
```

**Lines 26-28:** What the user sends to ask a question.
- `question: str` — The natural language question. Pydantic ensures this is present — a request without `question` returns a 422 error automatically.
- `chat_history: list[dict[str, str]] = []` — Previous conversation turns for follow-up questions. Each dict has `{"role": "user"|"assistant", "content": "..."}`. Defaults to empty list (no history), so this field is optional.

---

### QueryResponse

```
class QueryResponse(BaseModel):
    question: str
    answer: str
    sources: list[SourceReference]
    time_taken_seconds: float
```

**Lines 31-35:** The full answer returned to the user.
- `question` — Echoed back for logging/display purposes.
- `answer` — Claude's generated answer with inline citations.
- `sources: list[SourceReference]` — The structured source references (see above). This is a **nested model** — Pydantic automatically serializes the list of `SourceReference` objects into JSON arrays.
- `time_taken_seconds` — How long the query took. Useful for monitoring performance.

---

### DeleteResponse

```
class DeleteResponse(BaseModel):
    doc_id: str
    message: str
```

**Lines 38-40:** Confirmation after deleting a document and its chunks.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Pydantic BaseModel** | Data validation and serialization class |
| **Type Hints** | Python type annotations (`str`, `int`, `list[dict]`) that Pydantic enforces at runtime |
| **Optional Types** | `int \| None` means the field can be null in JSON |
| **Default Values** | `chat_history = []` makes the field optional in requests |
| **Nested Models** | `list[SourceReference]` — models inside models, auto-serialized |

| | |
|---|---|
| **Request/Response Pattern** | Separate models for input vs output, a REST API best practice |

## Why This Matters for Interviews

"I defined strict schemas for every API endpoint. This gives us three things for free: request validation (bad data is rejected with clear error messages), automatic JSON serialization, and auto-generated Swagger documentation. The nested `SourceReference` model shows how Pydantic handles complex response structures cleanly."

Section 4

# Embeddings

# embeddings.py — Text-to-Vector Conversion

**File:** `app/services/embeddings.py`

## What This File Does

Converts text into **vectors** (lists of numbers) that capture meaning. This is the core of semantic search — similar texts produce similar vectors, allowing us to find relevant chunks even when the exact words don't match.

## The Key Concept: What Are Embeddings?

Imagine you could represent any sentence as a point in space. Sentences with similar meanings would be close together, and unrelated sentences would be far apart.

```
"The cat sat on the mat"        → [0.12, -0.34, 0.56, ...]  (384 numbers)
"A cat was sitting on a rug"    → [0.13, -0.33, 0.55, ...]  (very close!)
"Quantum physics is complex"    → [-0.87, 0.45, -0.12, ...]  (very far away)
```

The `all-MiniLM-L6-v2` model was trained on millions of sentence pairs to learn this mapping. It converts any text into a 384-dimensional vector.

## Line-by-Line Explanation

```
from sentence_transformers import SentenceTransformer
```

**Line 1:** Imports the `SentenceTransformer` class from the `sentence-transformers` library. This library wraps Hugging Face transformer models and optimizes them specifically for generating embeddings (as opposed to text generation).

```
from app.config import settings
```

**Line 3:** Imports our centralized settings to get the model name (`all-MiniLM-L6-v2`).

```
_model = None
```

**Line 5:** Module-level variable to hold the loaded model. The underscore prefix (`_model`) is a Python convention meaning "private — don't import this directly."

```
def get_model() -> SentenceTransformer:
    global _model
    if _model is None:
        _model = SentenceTransformer(settings.embedding_model)
    return _model
```

**Lines 8-12: Lazy Singleton** pattern.
- `global _model` — Tells Python we're modifying the module-level `_model`, not creating a local variable.
- `if _model is None` — Only load the model on the FIRST call. Loading downloads weights (~80MB) and takes a few seconds. After that, it's cached in memory.
- `SentenceTransformer(settings.embedding_model)` — Loads the `all-MiniLM-L6-v2` model. On first run, it downloads from Hugging Face. After that, it loads from a local cache.
- **Why lazy?** If we loaded the model at import time (`_model = SentenceTransformer(...)`), it would slow down app startup and break tests that don't need embeddings.

```
def embed_texts(texts: list[str]) -> list[list[float]]:
    model = get_model()
    embeddings = model.encode(texts, show_progress_bar=False)
    return embeddings.tolist()
```

**Lines 15-18:** Converts multiple texts into vectors (used during document upload to embed all chunks at once).
- `texts: list[str]` — Input: a list of text chunks like `["Chapter 1...", "Chapter 2..."]`
- `model.encode(texts, ...)` — The model processes all texts in a single batch. This is much faster than encoding one at a time because the GPU/CPU can parallelize.
- `show_progress_bar=False` — Suppresses the tqdm progress bar (we're running in a server, not a notebook).
- `.tolist()` — Converts from NumPy array to plain Python list. ChromaDB expects Python lists, not NumPy arrays.
- **Return type:** `list[list[float]]` — A list of vectors. Each vector is a list of 384 floats.

```
def embed_query(query: str) -> list[float]:
    model = get_model()
    embedding = model.encode(query, show_progress_bar=False)
    return embedding.tolist()
```

**Lines 21-24:** Converts a single query into a vector (used at search time).
- Same logic as `embed_texts`, but for a single string.
- Returns a single vector: `list[float]` (384 numbers).

- **Why a separate function?** Clarity. `embed_texts` is for batch processing during upload, `embed_query` is for single queries during search. The underlying model call is the same.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Text Embeddings** | Converting text into numerical vectors that capture semantic meaning |
| **Sentence Transformers** | Library for generating high-quality text embeddings |
| **all-MiniLM-L6-v2** | A lightweight, fast embedding model producing 384-dim vectors |
| **Lazy Loading** | Model loads on first use, not at import time |
| **Singleton Pattern** | One model instance shared across all requests |
| **Batch Encoding** | Processing multiple texts at once for efficiency |
| **Vector Dimensions** | Each text becomes 384 numbers — the "coordinates" in meaning-space |

## Why This Matters for Interviews

"I chose open-source embeddings over OpenAI's API for three reasons: no extra API cost, works offline, and it demonstrates I understand the embedding layer rather than just making API calls. The lazy singleton ensures the model is only loaded once, since loading a transformer model takes several seconds."

Section 5

# Vector Store

# vector_store.py — ChromaDB Operations

**File:** `app/services/vector_store.py`

## What This File Does

Wraps all interactions with **ChromaDB**, our vector database. ChromaDB stores text chunks alongside their vector embeddings and metadata, enabling fast similarity search.

Think of it as a specialized database where instead of `SELECT * WHERE name = 'John'`, you say `"find the 5 chunks most similar to this vector"`.

## Line-by-Line Explanation

```
import chromadb


from app.config import settings
```

**Lines 1-3:** Import the ChromaDB client and our settings.

```
_client = None
_collection = None
```

**Lines 5-6:** Module-level singletons for the database client and collection. Same lazy-loading pattern as `embeddings.py`.

```
def get_collection() -> chromadb.Collection:
    global _client, _collection
    if _collection is None:
        _client = chromadb.PersistentClient(path=settings.chroma_persist_dir)
        _collection = _client.get_or_create_collection(
            name=settings.chroma_collection_name,
            metadata={"hnsw:space": "cosine"},
        )
    return _collection
```

**Lines 9-17:** Gets (or creates) the ChromaDB collection.
- `PersistentClient(path=...)` — Stores data on disk at `./chroma_db/`. Data survives server restarts. The alternative, `Client()`, is in-memory only (data lost on restart).
- `get_or_create_collection(...)` — If the collection "documents" exists, return it. If not, create it.

This is **idempotent** — safe to call multiple times.
- `metadata={"hnsw:space": "cosine"}` — **This is critical.** Tells ChromaDB to use **cosine similarity** to compare vectors.
- **Cosine similarity** measures the angle between two vectors (0 = identical direction, 1 = completely different).
- Alternative: Euclidean distance (measures straight-line distance). Cosine is preferred for text embeddings because it's invariant to vector magnitude — a longer document's embedding isn't penalized.
- **HNSW** = Hierarchical Navigable Small World graph. This is the algorithm ChromaDB uses internally for fast approximate nearest-neighbor search. It's O(log n) instead of O(n), making it fast even with millions of vectors.

---

### Adding Chunks

```
def add_chunks(
    ids: list[str],
    documents: list[str],
    embeddings: list[list[float]],
    metadatas: list[dict],
) -> None:
    collection = get_collection()
    collection.add(
        ids=ids,
        documents=documents,
        embeddings=embeddings,
        metadatas=metadatas,
    )
```

**Lines 20-32:** Stores chunks in ChromaDB. Each chunk gets four things:
- `ids` — Unique identifiers like `"abc123_0"`, `"abc123_1"` (doc_id + chunk_index).
- `documents` — The actual text of each chunk.
- `embeddings` — Pre-computed vectors (we compute them in `embeddings.py` rather than letting ChromaDB compute them, because we use our own model).
- `metadatas` — Dictionaries with `{doc_id, source_filename, page_number, chunk_index, ...}`. This metadata is what enables citations — when we retrieve a chunk, we know which file and page it came from.

---

### Querying Similar Chunks

```
def query_similar(
    query_embedding: list[float], k: int = settings.max_retrieval_k
) -> dict:
    collection = get_collection()
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=k,
        include=["documents", "metadatas", "distances"],
    )
    return results
```

**Lines 35-44:** The core **semantic search** operation.
- `query_embeddings=[query_embedding]` — The vector of the user's question. Wrapped in a list because ChromaDB supports batch queries.
- `n_results=k` — Return the top `k` (default 5) most similar chunks.
- `include=["documents", "metadatas", "distances"]` — What to return alongside the IDs. `distances` tells us how close each result was (lower = more similar with cosine).
- **What happens internally:** ChromaDB's HNSW index finds the `k` vectors closest to the query vector in 384-dimensional space. This is an approximate search — it's incredibly fast (milliseconds) but might miss the absolute closest vector in rare cases. The trade-off is worth it.
- **Return format:** `{"ids": [[...]], "documents": [[...]], "metadatas": [[...]], "distances": [[...]]}`. Everything is double-nested because ChromaDB supports batch queries.

---

### Deleting a Document

```
def delete_by_doc_id(doc_id: str) -> int:
    collection = get_collection()
    existing = collection.get(where={"doc_id": doc_id})
    count = len(existing["ids"])
    if count > 0:
        collection.delete(ids=existing["ids"])
    return count
```

**Lines 47-53:** Deletes all chunks belonging to a document.
- `collection.get(where={"doc_id": doc_id})` — **Metadata filtering.** This is why we stored `doc_id` in every chunk's metadata — it lets us find all chunks from a specific document.
- `collection.delete(ids=existing["ids"])` — ChromaDB requires deleting by ID, not by metadata filter. So we first find all matching IDs, then delete them.
- Returns the count of deleted chunks (used in the API response).

---

### Getting Chunks for a Document

```
def get_all_by_doc_id(doc_id: str) -> dict:
    collection = get_collection()
    return collection.get(where={"doc_id": doc_id}, include=["documents", "metadatas"])
```

**Lines 56-58:** Returns all chunks for a specific document. Used by the "View Chunks" feature in the UI.

---

### Listing All Documents

```
def get_all_documents_info() -> list[dict]:
    """Get summary info for all unique documents in the store."""
    collection = get_collection()
    all_data = collection.get(include=["metadatas"])

    docs = {}
    for metadata in all_data["metadatas"]:
        doc_id = metadata["doc_id"]
        if doc_id not in docs:
            docs[doc_id] = {
                "doc_id": doc_id,
                "filename": metadata["source_filename"],
                "num_chunks": 0,
                "num_pages": metadata.get("total_pages") if metadata.get("total_pages", -1) != -1 else No
            }
        docs[doc_id]["num_chunks"] += 1

    return list(docs.values())
```

**Lines 61-78:** Aggregates chunk-level data into document-level summaries.
- ChromaDB stores data per-chunk, not per-document. So to list documents, we iterate all chunks and group by `doc_id`.
- `num_chunks` is counted by incrementing for each chunk with that `doc_id`.
- `num_pages` handles the `-1` sentinel: we stored `-1` for non-PDF files (ChromaDB metadata must be strings, ints, or floats — not `None`), so we convert it back to `None` here.

---

### Getting All Chunks (for BM25)

```
def get_all_chunks() -> dict:
    """Get all chunks with documents and metadata (for BM25 index)."""
    collection = get_collection()
    return collection.get(include=["documents", "metadatas"])
```

**Lines 81-84:** Returns every chunk in the database. Used by `retriever.py` to build the BM25 keyword search index.

**Performance note:** This loads ALL chunks into memory on every query. For a portfolio project with a few documents, this is fine. In production with millions of chunks, you'd want a dedicated BM25 index (like Elasticsearch) that doesn't require loading everything.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Vector Database** | A database optimized for storing and searching high-dimensional vectors |
| **ChromaDB** | An open-source, lightweight vector database with Python-native API |
| **Cosine Similarity** | Measures angle between vectors — standard metric for text embeddings |
| **HNSW Algorithm** | Fast approximate nearest-neighbor search (what powers ChromaDB internally) |
| **Metadata Filtering** | Querying by stored attributes (doc_id, filename, etc.) |
| **Persistent Storage** | Data saved to disk, survives restarts |
| **Idempotent Operations** | `get_or_create_collection` is safe to call repeatedly |

## Why This Matters for Interviews

"I chose ChromaDB over FAISS because ChromaDB gives me metadata filtering — which I need for source citations — plus built-in persistence and a clean Python API. FAISS is a raw index without metadata support. I configured cosine similarity because it's invariant to vector magnitude, which matters when chunks have different lengths."

Section 6

# Document Processor

# document_processor.py — The Ingestion Pipeline

**File:** `app/services/document_processor.py`

## What This File Does

This is the **ingestion pipeline** — the complete journey from raw uploaded file to searchable chunks in ChromaDB. It handles file type detection, text extraction, chunking, embedding, and storage.

## Line-by-Line Explanation

```
import os
import uuid
import tempfile
```

**Lines 1-3:** Standard library imports.
- `os` — File path manipulation and temp file cleanup.
- `uuid` — Generates unique document IDs (UUID v4 = random, no collisions).
- `tempfile` — Creates temporary files for document loaders that need file paths.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader, TextLoader, Docx2txtLoader
```

**Lines 5-6:** LangChain's document loading and splitting tools.
- `RecursiveCharacterTextSplitter` — The most popular text splitter. It tries to split on paragraph boundaries first, then sentences, then words, then characters — preserving natural text boundaries.
- `PyPDFLoader` — Extracts text from PDFs, page by page.
- `TextLoader` — Reads plain text files.
- `Docx2txtLoader` — Extracts text from Microsoft Word documents.

```
from app.config import settings
from app.services import embeddings, vector_store
```

**Lines 8-9:** Our own modules — settings for chunk size/overlap, and the services to embed and store chunks.

```
SUPPORTED_EXTENSIONS = {".pdf", ".txt", ".docx"}
```

**Line 12:** A set of allowed file types. Using a set instead of a list gives O(1) lookup for the `in` check below. This is also our single source of truth for supported formats.

---

### The Main Processing Function

```
def process_upload(filename: str, file_bytes: bytes) -> dict:
    """Process an uploaded file: load, chunk, embed, store. Returns summary."""
    ext = os.path.splitext(filename)[1].lower()
    if ext not in SUPPORTED_EXTENSIONS:
        raise ValueError(f"Unsupported file type: {ext}. Supported: {SUPPORTED_EXTENSIONS}")
```

**Lines 15-19:** Validates the file type.
- `os.path.splitext("report.pdf")` returns `("report", ".pdf")` — we take `[1]` for the extension.
- `.lower()` — Handles `"REPORT.PDF"` gracefully.
- Raises `ValueError` for unsupported types — the router catches this and returns HTTP 400.

```
    # Write to temp file for loaders that need a file path
    with tempfile.NamedTemporaryFile(delete=False, suffix=ext) as tmp:
        tmp.write(file_bytes)
        tmp_path = tmp.name
```

**Lines 21-24:** Writes the uploaded bytes to a temporary file.
- **Why?** LangChain's document loaders expect file paths, not raw bytes. FastAPI gives us bytes from the upload.
- `delete=False` — We'll delete the file manually in the `finally` block (not automatically when the `with` block exits).
- `suffix=ext` — The temp file gets the right extension (`.pdf`), which some loaders need to detect format.

```
    try:
        # Load document
        if ext == ".pdf":
            loader = PyPDFLoader(tmp_path)
        elif ext == ".docx":
            loader = Docx2txtLoader(tmp_path)
        else:
            loader = TextLoader(tmp_path, encoding="utf-8")

        pages = loader.load()
```

**Lines 26-35:** Selects the right loader based on file type and extracts text.
- **PyPDFLoader** — Returns one `Document` object per page, with `metadata={"page": 0}`, `{"page": 1}`, etc. This is how we get page numbers for citations.
- **TextLoader** — Returns one `Document` for the entire file. `encoding="utf-8"` handles international characters.
- **Docx2txtLoader** — Returns one `Document` with all the text from the Word document.
- `pages = loader.load()` — Each loader returns a list of `Document` objects. Each `Document` has `.page_content` (the text) and `.metadata` (a dict with source info).

```
        # Chunk
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=settings.chunk_size,
            chunk_overlap=settings.chunk_overlap,
        )
        chunks = splitter.split_documents(pages)
```

**Lines 37-42:** Splits the loaded documents into smaller chunks.
- `RecursiveCharacterTextSplitter` works by trying these separators IN ORDER:
1. `"\n\n"` (paragraph breaks)
2. `"\n"` (line breaks)
3. `" "` (spaces/words)
4. `""` (individual characters — last resort)
- It tries to make each chunk ≤ `chunk_size` characters while splitting at the most natural boundary possible.
- `split_documents(pages)` — Takes the list of `Document` objects and returns a NEW list of smaller `Document` objects. Each chunk inherits the metadata from its parent page (including `page` number).

**Why chunk at all?** Three reasons:
1. **Precision** — Find the specific paragraph that answers the question, not a 10-page chapter.
2. **Token limits** — LLMs have context windows. Sending 5 small chunks is more efficient than 1 huge document.
3. **"Lost in the middle"** — Research shows LLMs pay less attention to information in the middle of long contexts. Shorter, focused chunks avoid this.

```
        # Determine page count
        num_pages = len(pages) if ext == ".pdf" else None
```

**Line 45:** PDFs have pages, text files don't. We track this for the API response.

```
        # Prepare data for vector store
        doc_id = str(uuid.uuid4())
        chunk_ids = []
        chunk_texts = []
        chunk_metadatas = []
```

**Lines 47-51:** Initializes the data structures for batch storage.
- `uuid.uuid4()` — Generates a random UUID like `"1e50be8d-77b5-414c-97f2-b19a04ed0b35"`. Virtually guaranteed unique (122 bits of randomness).

```
        for i, chunk in enumerate(chunks):
            chunk_ids.append(f"{doc_id}_{i}")
            chunk_texts.append(chunk.page_content)
            chunk_metadatas.append({
                "doc_id": doc_id,
                "source_filename": filename,
                "page_number": chunk.metadata.get("page", -1),
                "chunk_index": i,
                "total_chunks": len(chunks),
                "total_pages": num_pages if num_pages else -1,
            })
```

**Lines 53-63:** Builds parallel lists for ChromaDB.
- `chunk_ids` — Format: `"uuid_0"`, `"uuid_1"`, etc. Unique across all documents.
- `chunk.page_content` — The actual text of this chunk.
- **Metadata** — This is what enables all our features:
- `doc_id` — Groups chunks by document (for listing, deleting).
- `source_filename` — Shows in citations: `[Source: report.pdf, Page 3]`.
- `page_number` — From the PDF loader's metadata. `-1` for non-PDF files (ChromaDB doesn't support `None` in metadata).
- `chunk_index` — Ordering within the document.
- `total_chunks` / `total_pages` — Summary stats for the UI.

```
        # Embed and store
        chunk_embeddings = embeddings.embed_texts(chunk_texts)
        vector_store.add_chunks(chunk_ids, chunk_texts, chunk_embeddings, chunk_metadatas)
```

**Lines 65-67:** The final two steps.
1. `embed_texts(chunk_texts)` — Converts all chunk texts into vectors in one batch. For 10 chunks, this produces 10 vectors of 384 floats each.

2. `add_chunks(...)` — Stores everything in ChromaDB: the IDs, texts, vectors, and metadata.

```
        return {
            "doc_id": doc_id,
            "filename": filename,
            "num_chunks": len(chunks),
            "num_pages": num_pages,
        }

    finally:
        os.unlink(tmp_path)
```

**Lines 69-77:** Returns a summary and cleans up.
- `finally: os.unlink(tmp_path)` — Deletes the temporary file WHETHER OR NOT an error occurred. This prevents temp file buildup on disk.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Document Loading** | Extracting text from different file formats (PDF, TXT, DOCX) |
| **Text Chunking** | Splitting documents into smaller pieces for precise retrieval |
| **RecursiveCharacterTextSplitter** | Splits at natural boundaries (paragraphs > sentences > words) |
| **Chunk Overlap** | Shared characters between adjacent chunks to prevent information loss |
| **UUID** | Universally Unique Identifier for collision-free document IDs |
| **Temp Files** | Bridging between in-memory bytes and file-path-based loaders |
| **Batch Processing** | Embedding all chunks at once for efficiency |
| **Metadata Attachment** | Storing source info alongside each chunk for citations |

## Why This Matters for Interviews

"The ingestion pipeline is the foundation of any RAG system. I chose `RecursiveCharacterTextSplitter` because it preserves natural text boundaries — splitting on paragraphs first, then sentences, then words. The 200-character overlap prevents losing context at chunk boundaries. Each chunk carries metadata (filename, page number) which is what makes source citations possible."

Section 7

# Retriever

# retriever.py — Hybrid Search (BM25 + Semantic + RRF)

**File:** `app/services/retriever.py`

## What This File Does

This is the **most technically interesting file** in the project. It implements **hybrid search** — combining two fundamentally different search strategies and merging their results using Reciprocal Rank Fusion (RRF).

This is a real production pattern used by Google, Bing, and enterprise search systems. Most RAG tutorials only use semantic search, making this a strong differentiator.

## Why Hybrid Search?

| Query | Semantic Search | BM25 Keyword Search |
|---|---|---|
| "What are the findings?" | Matches "The results show..." (understands paraphrase) | Misses (no word overlap) |
| "ISM001-055 drug" | Might miss (it's a code, not a semantic concept) | Finds it (exact match) |
| "How does AI help doctors?" | Matches "AI assists physicians..." | Partial match only |

**Neither search is complete alone.** Semantic search understands meaning but misses exact terms. Keyword search finds exact terms but misses paraphrases. Combining them gives the best of both worlds.

## Line-by-Line Explanation

```
from rank_bm25 import BM25Okapi

from app.config import settings
from app.services import embeddings, vector_store
```

**Lines 1-4:**
- `BM25Okapi` — Implementation of the **BM25 algorithm** (Best Match 25), the gold standard for keyword search. "Okapi" refers to the Okapi information retrieval system where BM25 was first developed.
- We import both `embeddings` (for semantic search) and `vector_store` (for ChromaDB operations).

```
def _tokenize(text: str) -> list[str]:
    """Simple whitespace tokenization with lowercasing."""
    return text.lower().split()
```

**Lines 7-9:** Converts text into tokens (words) for BM25.
- `.lower()` — Makes search case-insensitive ("AI" matches "ai").
- `.split()` — Splits on whitespace. This is simple but effective. Production systems might use stemming (reducing "running" to "run") or lemmatization, but for a portfolio project, whitespace tokenization demonstrates the concept clearly.

---

### The Main Hybrid Search Function

```
def hybrid_search(query: str, k: int = settings.max_retrieval_k) -> dict:
```

**Line 12:** Entry point. Takes a question and returns the top `k` chunks.

#### Step 1: Semantic Search

```
    # --- Semantic search ---
    query_embedding = embeddings.embed_query(query)
    semantic_results = vector_store.query_similar(query_embedding, k=k * 2)

    semantic_ids = semantic_results["ids"][0] if semantic_results["ids"] else []
    semantic_docs = semantic_results["documents"][0] if semantic_results["documents"] else []
    semantic_metas = semantic_results["metadatas"][0] if semantic_results["metadatas"] else []
```

**Lines 17-23:**
1. Convert the question into a vector.
2. Find the `k * 2` most similar chunks in ChromaDB. **Why `k * 2`?** We retrieve more candidates than needed because after merging with BM25, some semantic results might be pushed out. Having extra candidates gives RRF more data to work with.
3. Extract the IDs, documents, and metadata from ChromaDB's nested response format.

#### Step 2: BM25 Keyword Search

```
    # --- BM25 keyword search ---
    all_chunks = vector_store.get_all_chunks()
    all_ids = all_chunks["ids"]
    all_docs = all_chunks["documents"]
```

```
    all_metas = all_chunks["metadatas"]

    if not all_docs:
        return {"documents": [[]], "metadatas": [[]], "ids": [[]]}
```

**Lines 25-32:** Load all chunks from ChromaDB for BM25 scoring.
- BM25 needs access to the entire corpus to calculate term frequency and inverse document frequency. There's no shortcut — it must see all documents.
- Empty check: if no documents exist, return empty results.

```
    tokenized_corpus = [_tokenize(doc) for doc in all_docs]
    bm25 = BM25Okapi(tokenized_corpus)
    bm25_scores = bm25.get_scores(_tokenize(query))
```

**Lines 34-36:** Build and query the BM25 index.
- `tokenized_corpus` — Each document becomes a list of words: `[["chapter", "1", "introduction", ...], ...]`
- `BM25Okapi(tokenized_corpus)` — Builds the BM25 index. Internally, it calculates:
- **TF (Term Frequency):** How often each word appears in each document.
- **IDF (Inverse Document Frequency):** How rare each word is across ALL documents. Rare words (like "pharmaceuticals") score higher than common words (like "the").
- **Document Length Normalization:** Longer documents are penalized to avoid bias.
- `bm25.get_scores(...)` — Returns a score for EVERY document in the corpus. Higher score = more relevant keywords.

```
    # Get top BM25 results
    bm25_ranked = sorted(
        enumerate(bm25_scores), key=lambda x: x[1], reverse=True
    )[:k * 2]

    bm25_ids = [all_ids[idx] for idx, _ in bm25_ranked]
```

**Lines 39-43:** Sort all chunks by BM25 score and take the top `k * 2`.
- `enumerate(bm25_scores)` — Pairs each score with its index: `[(0, 0.5), (1, 2.3), (2, 0.1), ...]`
- `sorted(..., reverse=True)` — Highest scores first.
- `[:k * 2]` — Take the top candidates (same number as semantic search).

**Step 3: Reciprocal Rank Fusion (RRF)**

```
    # --- Reciprocal Rank Fusion ---
    rrf_constant = 60  # Standard RRF constant
    scores = {}
```

**Lines 45-47:**
- `rrf_constant = 60` — The standard constant from the original RRF paper. It prevents top-ranked results from dominating too aggressively. A value of 60 means that rank 1 gets a score of `1/61 = 0.0164`, while rank 10 gets `1/70 = 0.0143` — a smooth decay rather than a cliff.

```
    # Score semantic results
    for rank, chunk_id in enumerate(semantic_ids):
        scores[chunk_id] = scores.get(chunk_id, 0) + (
            settings.semantic_weight / (rrf_constant + rank + 1)
        )
```

**Lines 49-53:** Score each semantic result.
- **RRF formula:** `score = weight / (k + rank + 1)`
- Rank 0 (best match): `0.7 / (60 + 0 + 1) = 0.7 / 61 = 0.01148`
- Rank 1: `0.7 / (60 + 1 + 1) = 0.7 / 62 = 0.01129`
- Rank 9: `0.7 / (60 + 9 + 1) = 0.7 / 70 = 0.01000`
- `settings.semantic_weight` is `0.7` — semantic search gets 70% influence.

```
    # Score BM25 results
    for rank, chunk_id in enumerate(bm25_ids):
        scores[chunk_id] = scores.get(chunk_id, 0) + (
            settings.keyword_weight / (rrf_constant + rank + 1)
        )
```

**Lines 55-59:** Same formula for BM25 results, but with `keyword_weight = 0.3`.
- **The magic of RRF:** If a chunk appears in BOTH semantic AND BM25 results, its scores ADD UP. A chunk ranked #1 in both lists gets:
- Semantic: `0.7 / 61 = 0.01148`
- BM25: `0.3 / 61 = 0.00492`
- **Total: 0.01640** (much higher than a chunk in only one list)
- This naturally boosts chunks that both search strategies agree are relevant.

```
    # Sort by fused score and take top k
    top_ids = sorted(scores, key=scores.get, reverse=True)[:k]
```

**Line 62:** Final ranking. The `k` chunks with the highest combined RRF scores win.

**Step 4: Assemble Results**

```
    # Build a lookup from all available data
    id_to_doc = {}
    id_to_meta = {}

    for i, chunk_id in enumerate(semantic_ids):
        id_to_doc[chunk_id] = semantic_docs[i]
        id_to_meta[chunk_id] = semantic_metas[i]

    for i, chunk_id in enumerate(all_ids):
        if chunk_id not in id_to_doc:
            id_to_doc[chunk_id] = all_docs[i]
            id_to_meta[chunk_id] = all_metas[i]

    # Assemble final results in ChromaDB return format
    final_docs = [id_to_doc[cid] for cid in top_ids if cid in id_to_doc]
    final_metas = [id_to_meta[cid] for cid in top_ids if cid in id_to_meta]
    final_ids = [cid for cid in top_ids if cid in id_to_doc]

    return {
        "documents": [final_docs],
        "metadatas": [final_metas],
        "ids": [final_ids],
    }
```

**Lines 64-86:** Maps the winning IDs back to their full text and metadata.
- We build lookup dictionaries from both the semantic results and the full corpus (some BM25 winners might not be in the semantic results).
- Return format matches ChromaDB's output format (double-nested lists) so `qa_chain.py` doesn't need to know whether results came from simple or hybrid search.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Hybrid Search** | Combining multiple search strategies for better results |

| | |
|---|---|
| **BM25 (Okapi)** | The standard algorithm for keyword-based document retrieval |
| **Term Frequency (TF)** | How often a word appears in a document |
| **Inverse Document Frequency (IDF)** | How rare a word is across all documents (rare = more important) |
| **Semantic Search** | Finding documents by meaning similarity using vector embeddings |
| **Reciprocal Rank Fusion (RRF)** | A method to combine ranked lists from different search systems |
| **Weighted Scoring** | Giving different importance to different search methods (70/30) |
| **Candidate Over-retrieval** | Fetching `k*2` candidates from each method before fusing to `k` |

## Why This Matters for Interviews

"I implemented hybrid search because pure semantic search fails on exact terms — product names, acronyms, codes — while pure keyword search fails on paraphrased questions. I combine them using Reciprocal Rank Fusion with a 70/30 weighting. RRF is elegant because it doesn't need score normalization — it works purely on rank positions. Chunks that both methods agree on get naturally boosted. This is the same approach Google and other production search systems use."

Section 8

# Qa Chain

# qa_chain.py — Claude RAG Chain with Citations

**File:** `app/services/qa_chain.py`

## What This File Does

This is where **retrieval meets generation**. It takes the retrieved chunks from hybrid search, formats them into a prompt, sends that prompt to Claude, and returns the answer with structured source citations. This is the "G" in RAG.

## Line-by-Line Explanation

```
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate

from app.config import settings
from app.services import retriever
```

**Lines 1-5:**
- `ChatAnthropic` — LangChain's wrapper around the Anthropic API. Handles API calls, retries, and response parsing.
- `ChatPromptTemplate` — A template for building prompts with variable placeholders (`{context}`, `{question}`).
- `retriever` — Our hybrid search module (semantic + BM25 + RRF).

---

### The Prompt Template

```
QA_PROMPT = ChatPromptTemplate.from_template(
    """You are a document Q&A assistant. Answer the question based ONLY on the
provided context excerpts. For each claim in your answer, cite the source
using the format [Source: filename, Page X].

If the context does not contain enough information to answer the question,
explicitly state that the provided documents don't contain this information.

Context:
{context}

Chat History:
```

```
{chat_history}

Question: {question}"""
)
```

**Lines 7-22:** The **prompt template** — arguably the most important piece of prompt engineering in the project.

Breaking down each instruction:

> **"Answer based ONLY on the provided context excerpts"** — This is the anti-hallucination instruction. Without it, Claude might use its training knowledge to answer, defeating the purpose of RAG. The word "ONLY" is intentionally capitalized for emphasis.
>
> **"For each claim, cite the source using the format [Source: filename, Page X]"** — Forces structured citations. By specifying the exact format, we ensure citations are consistent and parseable. Claude follows formatting instructions reliably.
>
> **"If the context does not contain enough information... explicitly state that"** — The honesty instruction. Without this, Claude might try to be helpful by guessing, which breaks trust. A good RAG system says "I don't know" rather than fabricating an answer.
>
> `{context}` — Replaced with the formatted chunks (see `format_context` below).
>
> `{chat_history}` — Replaced with previous Q&A turns for follow-up questions.
>
> `{question}` — The user's actual question.

**Why is the prompt defined as a module-level constant?** It never changes at runtime, and defining it once makes it easy to find and modify.

---

### LLM Initialization

```
_llm = None


def get_llm() -> ChatAnthropic:
    global _llm
    if _llm is None:
        if not settings.anthropic_api_key:
            raise RuntimeError("ANTHROPIC_API_KEY is not set. Add it to your .env file.")
        _llm = ChatAnthropic(
            model=settings.llm_model,
```

```
            anthropic_api_key=settings.anthropic_api_key,
            temperature=settings.llm_temperature,
        )
    return _llm
```

**Lines 24-37:** Lazy singleton for the Claude LLM client.
- `if not settings.anthropic_api_key` — Explicit check with a clear error message. Since we made the API key optional in config (for tests), we validate here where it's actually needed.
- `temperature=0` — Deterministic output. Same question + same context = same answer. For factual Q&A, you want reproducibility, not creativity.

---

### Formatting Context

```
def format_context(results: dict) -> tuple[str, list[dict]]:
    """Format retrieved chunks into context string and source references."""
    context_parts = []
    sources = []

    documents = results["documents"][0] if results["documents"] else []
    metadatas = results["metadatas"][0] if results["metadatas"] else []
```

**Lines 40-46:** Unpacks the retriever's results. The `[0]` indexing handles ChromaDB's double-nested format.

```
    for doc, metadata in zip(documents, metadatas):
        page = metadata.get("page_number", -1)
        page_display = page + 1 if page >= 0 else None
        filename = metadata.get("source_filename", "unknown")

        page_label = f"Page {page_display}" if page_display else "N/A"
        context_parts.append(
            f"[Source: {filename}, {page_label}]\n{doc}"
        )
```

**Lines 48-56:** Formats each chunk for the prompt.
- `page + 1` — Converts from 0-indexed (internal) to 1-indexed (human-friendly). Page 0 becomes "Page 1".
- Each chunk in the context looks like:
`[Source: report.pdf, Page 3] AI-powered diagnostic tools have achieved remarkable`

`accuracy...`

- The `[Source: ...]` prefix matches what we asked Claude to cite. This makes it easy for Claude to copy the citation format exactly.

```
        sources.append({
            "filename": filename,
            "page": page_display,
            "chunk_index": metadata.get("chunk_index", 0),
            "snippet": doc[:200] + "..." if len(doc) > 200 else doc,
        })

    return "\n\n---\n\n".join(context_parts), sources
```

**Lines 58-65:** Builds the structured source references for the API response.

- `snippet: doc[:200] + "..."` — Shows the first 200 characters as a preview. Enough to identify the source without flooding the response.
- Chunks are separated by `---` in the context string — a visual separator that helps Claude distinguish between different sources.
- Returns TWO things: the formatted context string (for the prompt) and the sources list (for the API response).

---

### Chat History Formatting

```
def format_chat_history(chat_history: list[dict[str, str]]) -> str:
    if not chat_history:
        return "No previous conversation."
    lines = []
    for msg in chat_history[-5:]:  # Keep last 5 turns
        role = msg.get("role", "user")
        content = msg.get("content", "")
        lines.append(f"{role.capitalize()}: {content}")
    return "\n".join(lines)
```

**Lines 68-76:** Formats previous conversation turns for follow-up questions.

- `chat_history[-5:]` — Only includes the last 5 turns. This prevents the prompt from growing too large with long conversations.
- `"No previous conversation."` — Explicit text for the empty case. Leaving it blank might confuse the prompt template.
- This enables conversations like:
- Q: "What does chapter 3 cover?" → A: "Chapter 3 covers drug discovery..."

- Q: "How long does the traditional process take?" → Claude knows "the process" refers to drug discovery from the chat history.

---

### The Main Answer Function

```
def answer_question(question: str, chat_history: list[dict[str, str]] | None = None) -> dict:
    """Retrieve relevant chunks and generate an answer with citations."""
    if chat_history is None:
        chat_history = []
```

**Lines 79-82:** Entry point. Accepts the question and optional chat history.

```
    # Hybrid search: semantic + BM25 with Reciprocal Rank Fusion
    results = retriever.hybrid_search(question, k=settings.max_retrieval_k)
```

**Lines 84-85: Retrieval step.** Calls the hybrid retriever to find the top 5 most relevant chunks. This is the "R" in RAG.

```
    context_str, sources = format_context(results)

    if not context_str:
        return {
            "answer": "No documents have been uploaded yet. Please upload a document first.",
            "sources": [],
        }
```

**Lines 87-93:** Formats the results and handles the empty case. If no documents exist, return a helpful message instead of sending an empty context to Claude.

```
    # Generate answer with Claude
    llm = get_llm()
    chain = QA_PROMPT | llm
    response = chain.invoke({
        "context": context_str,
        "chat_history": format_chat_history(chat_history),
        "question": question,
    })
```

**Lines 95-102: Generation step.** The "G" in RAG.
- `QA_PROMPT | llm` — This is **LangChain Expression Language (LCEL)**. The `|` operator creates a chain: the prompt template feeds into the LLM. It's like a Unix pipe: `template | model`.
- `chain.invoke({...})` — Fills in the template variables and sends the complete prompt to Claude.
- Claude receives a prompt like:
```

You are a document Q&A assistant. Answer based ONLY on...

Context:
[Source: report.pdf, Page 2]
AI-powered diagnostic tools have achieved...

---

[Source: report.pdf, Page 3]
Traditional drug discovery takes 10-15 years...

Chat History:
No previous conversation.

Question: What is the accuracy of AI in detecting lung nodules?
```

```
    return {
        "answer": response.content,
        "sources": sources,
    }
```

**Lines 104-107:** Returns the answer text and source references. `response.content` extracts just the text from Claude's response object.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **RAG (Retrieval-Augmented Generation)** | Retrieve relevant context, then generate an answer from it |
| **Prompt Engineering** | Crafting instructions that guide the LLM's behavior (anti-hallucination, citation format, honesty) |
| **LCEL (LangChain Expression Language)** | Composable chains using the `\|` pipe operator |
| **Grounded Generation** | Generating answers strictly from provided context, not training data |

| | |
|---|---|
| **Source Citations** | Tracing each claim back to its source document and page |
| **Conversation Memory** | Including chat history for follow-up questions |
| **Temperature** | Controls randomness; 0 = deterministic output |
| **Context Window** | The amount of text an LLM can process at once |

## Why This Matters for Interviews

"The prompt template is where RAG lives or dies. Three key instructions: answer ONLY from context (prevents hallucination), cite every claim (builds trust), and say 'I don't know' when context is insufficient (honesty over helpfulness). I use LCEL to compose the prompt and LLM into a clean chain. Temperature is 0 for deterministic, reproducible answers."

Section 9

# Router Documents

# routers/documents.py — Document Management API

**File:** `app/routers/documents.py`

## What This File Does

Defines all API endpoints for managing documents: uploading, listing, deleting, and inspecting chunks. This is a **FastAPI Router** — a way to group related endpoints and mount them under a URL prefix.

## Line-by-Line Explanation

```
from fastapi import APIRouter, UploadFile, File, HTTPException

from app.models import UploadResponse, DocumentInfo, DeleteResponse
from app.services import document_processor, vector_store
```

**Lines 1-4:**
- `APIRouter` — Groups endpoints under a shared prefix and tag.
- `UploadFile` — FastAPI's abstraction for uploaded files. Provides `.filename`, `.read()`, `.content_type`, etc.
- `File(...)` — A dependency that tells FastAPI this parameter comes from a `multipart/form-data` upload.
- `HTTPException` — Raises HTTP error responses with status codes and messages.

```
router = APIRouter(prefix="/documents", tags=["Documents"])
```

**Line 6:** Creates a router where all endpoints start with `/documents`.
- `prefix="/documents"` — A `@router.post("/upload")` becomes `POST /documents/upload`.
- `tags=["Documents"]` — Groups these endpoints in the Swagger UI under a "Documents" section.

---

### Upload Endpoint

```
@router.post("/upload", response_model=UploadResponse)
async def upload_document(file: UploadFile = File(...)):
    """Upload a PDF, TXT, or DOCX file to the document store."""
    if not file.filename:
        raise HTTPException(status_code=400, detail="No filename provided")
```

**Lines 9-13:**
- `response_model=UploadResponse` — FastAPI validates the return value against this Pydantic model and generates the response schema in Swagger docs.
- `async def` — The function is asynchronous because `file.read()` is an async operation (reading from a network stream).
- `file: UploadFile = File(...)` — FastAPI parses the multipart upload and provides the file object. `File(...)` means it's required.
- Validates that a filename exists (edge case: some HTTP clients might send empty filenames).

```
    try:
        file_bytes = await file.read()
        result = document_processor.process_upload(file.filename, file_bytes)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Processing error: {str(e)}")
```

**Lines 15-21:**
- `await file.read()` — Reads the entire uploaded file into memory as `bytes`. The `await` is necessary because `UploadFile.read()` is async.
- `document_processor.process_upload(...)` — Calls our ingestion pipeline (load → chunk → embed → store).
- **Error handling:** `ValueError` (unsupported file type) → 400 Bad Request. Any other error → 500 Internal Server Error.

```
    return UploadResponse(
        doc_id=result["doc_id"],
        filename=result["filename"],
        num_chunks=result["num_chunks"],
        num_pages=result["num_pages"],
        message=f"Successfully processed '{result['filename']}' into {result['num_chunks']} chunks",
    )
```

**Lines 23-29:** Returns a structured response. FastAPI automatically serializes this Pydantic model to JSON.

---

### List Documents Endpoint

```
@router.get("/", response_model=list[DocumentInfo])
def list_documents():
    """List all uploaded documents."""
    docs = vector_store.get_all_documents_info()
    return [DocumentInfo(**doc) for doc in docs]
```

**Lines 32-36:**

- `response_model=list[DocumentInfo]` — Returns a JSON array of document summaries.
- `DocumentInfo(**doc)` — The `**` unpacks the dictionary into keyword arguments: `DocumentInfo(doc_id="abc", filename="report.pdf", ...)`. This is called **dictionary unpacking**.
- `def` (not `async def`) — No async operations needed here.

---

### Delete Document Endpoint

```
@router.delete("/{doc_id}", response_model=DeleteResponse)
def delete_document(doc_id: str):
    """Delete a document and all its chunks from the store."""
    deleted_count = vector_store.delete_by_doc_id(doc_id)
    if deleted_count == 0:
        raise HTTPException(status_code=404, detail=f"Document '{doc_id}' not found")

    return DeleteResponse(
        doc_id=doc_id,
        message=f"Deleted {deleted_count} chunks",
    )
```

**Lines 39-49:**

- `"/{doc_id}"` — **Path parameter.** The URL `DELETE /documents/abc-123` extracts `doc_id="abc-123"`.
- Returns 404 if no chunks were found with that `doc_id`. This handles the case where someone tries to delete an already-deleted or non-existent document.

---

### Chunk Inspection Endpoint

```
@router.get("/{doc_id}/chunks")
def get_document_chunks(doc_id: str):
    """Get chunk details for a specific document (for the insights panel)."""
    data = vector_store.get_all_by_doc_id(doc_id)
```

```
    if not data["ids"]:
        raise HTTPException(status_code=404, detail=f"Document '{doc_id}' not found")

    chunks = []
    for i, (doc, metadata) in enumerate(zip(data["documents"], data["metadatas"])):
        chunks.append({
            "chunk_index": metadata.get("chunk_index", i),
            "page_number": metadata.get("page_number", -1),
            "text": doc,
            "length": len(doc),
        })

    return {
        "doc_id": doc_id,
        "filename": data["metadatas"][0]["source_filename"],
        "total_chunks": len(chunks),
        "chunks": sorted(chunks, key=lambda c: c["chunk_index"]),
    }
```

**Lines 52-73:** Returns all chunks for a document — used by the "View Chunks" feature in the Streamlit sidebar.
- `zip(data["documents"], data["metadatas"])` — Iterates documents and metadata in parallel.
- `sorted(..., key=lambda c: c["chunk_index"])` — Returns chunks in order (ChromaDB doesn't guarantee ordering).
- Each chunk includes its full text and character length, so the UI can show chunk size distribution.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **FastAPI Router** | Groups related endpoints under a prefix with shared tags |
| **Path Parameters** | URL segments like `/{doc_id}` extracted as function arguments |
| **File Upload** | `UploadFile` + `File(...)` for handling multipart form data |
| **Async/Await** | Non-blocking I/O for reading uploaded files |

| | |
|---|---|
| **HTTP Status Codes** | 200 (success), 400 (bad request), 404 (not found), 500 (server error) |
| **Response Models** | Pydantic models that validate and document API responses |
| **REST Conventions** | POST for create, GET for read, DELETE for delete |
| **Dictionary Unpacking** | `**dict` to pass dictionary values as keyword arguments |

## Why This Matters for Interviews

"I structured the API following REST conventions — POST for upload, GET for listing, DELETE for removal. The router groups all document operations under `/documents` with proper error handling: 400 for bad input, 404 for missing resources, 500 for server errors. The chunk inspection endpoint lets users see exactly how their document was processed, which builds transparency and trust."

Section 10

# Router Query

# routers/query.py — Query API Endpoint

**File:** `app/routers/query.py`

## What This File Does

Defines the single most important endpoint in the entire API: `POST /query`. This is where users ask questions and get answers. It's intentionally simple — all the complexity is in the services it calls.

## Line-by-Line Explanation

```
import time

from fastapi import APIRouter, HTTPException

from app.models import QueryRequest, QueryResponse, SourceReference
from app.services import qa_chain
```

**Lines 1-6:**
- `time` — For measuring query duration.
- `QueryRequest` — Validates the incoming question and optional chat history.
- `QueryResponse` — Structures the answer with sources and timing.
- `SourceReference` — Individual source citation model.
- `qa_chain` — The service that does the actual retrieval + generation.

```
router = APIRouter(tags=["Query"])
```

**Line 8:** Router without a prefix — the endpoint will be at `/query` (not `/query/query`). Tagged "Query" for Swagger docs grouping.

```
@router.post("/query", response_model=QueryResponse)
def query_documents(request: QueryRequest):
    """Ask a question about your uploaded documents."""
    if not request.question.strip():
        raise HTTPException(status_code=400, detail="Question cannot be empty")
```

**Lines 11-15:**
- `request: QueryRequest` — FastAPI automatically parses the JSON body into a `QueryRequest` object. If the JSON is malformed or missing `question`, FastAPI returns 422 automatically.
- `.strip()` — Catches questions that are only whitespace (like `" "`). The Pydantic model ensures

`question` exists, but we additionally check it's not blank.

```
    start_time = time.time()

    try:
        result = qa_chain.answer_question(request.question, request.chat_history)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Query error: {str(e)}")

    time_taken = round(time.time() - start_time, 2)
```

**Lines 17-24:**
- `time.time()` — Records the start time in seconds.
- `qa_chain.answer_question(...)` — Calls the full RAG pipeline: hybrid search → context formatting → Claude generation.
- `round(..., 2)` — Rounds to 2 decimal places (e.g., `4.23` seconds).
- The timing includes: embedding the query, searching ChromaDB, running BM25, fusing results, and waiting for Claude's response. Typically 2-5 seconds, mostly spent waiting for Claude.

```
    sources = [SourceReference(**s) for s in result["sources"]]

    return QueryResponse(
        question=request.question,
        answer=result["answer"],
        sources=sources,
        time_taken_seconds=time_taken,
    )
```

**Lines 26-33:**
- `[SourceReference(**s) for s in result["sources"]]` — Converts raw dictionaries from `qa_chain` into validated Pydantic models. This ensures the response schema is exactly what we promised in `QueryResponse`.
- The response includes everything the frontend needs: the question (echoed back), the answer, structured sources, and timing.

## Design Note: Why This File Is So Small

This router is intentionally thin. It handles only HTTP concerns:
- Input validation (empty question check)
- Error handling (try/except → HTTPException)
- Response formatting (dict → Pydantic model)

- Timing

All business logic lives in `qa_chain.py`. This separation is called the **"thin controller" pattern** — routers handle HTTP, services handle logic. Benefits:
- You can test `qa_chain.answer_question()` directly without HTTP.
- You could add a CLI interface that calls `qa_chain` without any router changes.
- The router stays readable at a glance.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Thin Controller** | Router handles HTTP only; logic lives in services |
| **Request Body Parsing** | FastAPI auto-parses JSON body into Pydantic model |
| **Response Timing** | Measuring end-to-end query performance |
| **Error Propagation** | Service exceptions → HTTP 500 with error message |

## Why This Matters for Interviews

"The query router follows the thin controller pattern — it only handles HTTP concerns like validation, error mapping, and timing. All the RAG logic lives in the service layer. This means I can test the retrieval pipeline independently and could easily add other interfaces (CLI, WebSocket) without duplicating business logic."

Section 11

# Main

# main.py — FastAPI App Assembly

**File:** `app/main.py`

## What This File Does

The entry point for the entire backend. Creates the FastAPI application, adds middleware, and mounts the routers. This is what `uvicorn app.main:app` points to.

## Line-by-Line Explanation

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.routers import documents, query
```

**Lines 1-4:**
- `FastAPI` — The main application class.
- `CORSMiddleware` — Handles Cross-Origin Resource Sharing (explained below).
- We import our two routers to mount them on the app.

```
app = FastAPI(
    title="RAG Document Q&A",
    description="Upload documents and ask questions — powered by Claude AI and semantic search",
    version="1.0.0",
)
```

**Lines 6-10:** Creates the app instance.
- `title`, `description`, `version` — These appear in the auto-generated Swagger documentation at `/docs`. They make the API self-documenting.
- This is the object that `uvicorn` looks for when you run `uvicorn app.main:app`.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

**Lines 12-18:** Adds **CORS (Cross-Origin Resource Sharing)** middleware.

**What is CORS?** Browsers block requests from one domain to another by default. If your Streamlit UI runs on `localhost:8501` and the API runs on `localhost:8000`, the browser considers these different "origins" and blocks the request.

CORS middleware tells the browser "it's okay, these origins are allowed to talk to me."

- `allow_origins=["*"]` — Allow requests from ANY origin. In production, you'd restrict this to your actual frontend domain.
- `allow_methods=["*"]` — Allow all HTTP methods (GET, POST, DELETE, etc.).
- `allow_headers=["*"]` — Allow all headers (including `Content-Type`).
- **Why add it?** Without CORS, the Streamlit UI's `requests.post()` would be blocked by the browser's security policy.

```
app.include_router(documents.router)
app.include_router(query.router)
```

**Lines 20-21:** Mounts the routers on the app.
- `documents.router` — Adds all `/documents/*` endpoints.
- `query.router` — Adds the `/query` endpoint.
- This is why routers exist — you can organize endpoints in separate files and mount them all in one place.

```
@app.get("/health")
def health_check():
    return {"status": "ok", "message": "RAG Document Q&A is running"}
```

**Lines 24-26:** Health check endpoint. Used by:
- Deployment platforms (Render, Railway) to know the app is alive.
- Monitoring systems to detect outages.
- The Streamlit UI could use this to show connection status.

```
@app.get("/")
def root():
    return {
        "message": "Welcome to RAG Document Q&A",
        "docs": "Visit /docs to try the API interactively",
        "endpoints": {
            "health": "GET /health",
```

```
            "upload": "POST /documents/upload",
            "list_docs": "GET /documents/",
            "delete_doc": "DELETE /documents/{doc_id}",
            "query": "POST /query",
        },
    }
```

**Lines 29-41:** Root endpoint — a quick reference for anyone hitting the base URL. Lists all available endpoints so developers don't have to dig through docs.

## Concepts Covered

| Concept | What It Is |
|---|---|
| **FastAPI Application** | The central app object that handles all HTTP requests |
| **CORS Middleware** | Allows cross-origin requests (frontend → backend on different ports) |
| **Router Mounting** | Attaching groups of endpoints to the main app |
| **Health Check** | An endpoint for monitoring and deployment readiness |
| **Swagger/OpenAPI** | Auto-generated API documentation at `/docs` |

## Why This Matters for Interviews

"The main file is deliberately minimal — it only configures the app and mounts routers. CORS middleware enables the Streamlit frontend to communicate with the API across different ports. The health check endpoint is essential for deployment platforms to know the service is alive."

Section 12

# Streamlit App

# streamlit_app.py — Chat UI Frontend

**File:** `ui/streamlit_app.py`

## What This File Does

The user-facing frontend. Provides a chat interface for asking questions, a sidebar for managing documents, and expandable sections for source citations and chunk inspection. All communication with the backend happens through HTTP requests.

## Line-by-Line Explanation

### Setup

```
import streamlit as st
import requests
import os

API_URL = os.getenv("API_URL", "http://localhost:8000")
```

**Lines 1-5:**
- `streamlit as st` — Streamlit's main module. Every UI element (`st.title`, `st.chat_input`, etc.) comes from here.
- `requests` — Python's standard HTTP library for calling the FastAPI backend.
- `API_URL` — Reads from environment variable, defaults to localhost. This makes the UI configurable: `API_URL=http://api:8000` for Docker, `http://localhost:8080` for custom ports.

```
st.set_page_config(
    page_title="RAG Document Q&A",
    page_icon="■",
    layout="wide",
)
```

**Lines 7-11:** Configures the browser tab title, favicon, and page width. `layout="wide"` uses the full browser width instead of a centered narrow column — needed for the sidebar + chat layout.

---

### Session State

```
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []
if "messages" not in st.session_state:
    st.session_state.messages = []
```

**Lines 13-17:** Initializes persistent state.

**Why do we need this?** Streamlit reruns the ENTIRE script from top to bottom on every interaction (button click, text input, etc.). Without `st.session_state`, all variables would reset. Session state persists across reruns.

- `chat_history` — The Q&A pairs sent to the backend for conversation context (`[{"role": "user", "content": "..."}, ...]`).
- `messages` — The full message list for rendering the chat UI (includes sources and formatting).

---

**Sidebar: Document Upload**

```
with st.sidebar:
    st.header("■ Document Library")

    uploaded_file = st.file_uploader(
        "Upload a document",
        type=["pdf", "txt", "docx"],
        help="Supported formats: PDF, TXT, DOCX",
    )
```

**Lines 19-28:**
- `with st.sidebar:` — Everything inside this block appears in the left sidebar.
- `st.file_uploader(...)` — Renders a file upload widget. `type=[...]` restricts to allowed formats. Streamlit handles the file picker dialog.

```
    if uploaded_file and st.button("Process Document", type="primary", use_container_width=True):
        with st.spinner(f"Processing '{uploaded_file.name}'..."):
            try:
                files = {"file": (uploaded_file.name, uploaded_file.getvalue())}
                response = requests.post(f"{API_URL}/documents/upload", files=files, timeout=120)
```

**Lines 30-34:**
- `if uploaded_file and st.button(...)` — Only show the process button when a file is selected.

Both conditions must be true.
- `st.spinner(...)` — Shows a loading animation while processing.
- `files = {"file": (filename, bytes)}` — Constructs a multipart form upload. The `requests` library sends this as `multipart/form-data`, which is what FastAPI's `UploadFile` expects.
- `timeout=120` — 2 minutes. Document processing can take a while (embedding all chunks).

```
                if response.status_code == 200:
                    data = response.json()
                    st.success(f"■ {data['message']}")
                else:
                    st.error(f"Error: {response.json().get('detail', 'Upload failed')}")
            except requests.exceptions.ConnectionError:
                st.error("Cannot connect to the API. Is the backend running?")
```

**Lines 35-41:** Handles the response.
- `st.success(...)` — Green banner for success.
- `st.error(...)` — Red banner for errors.
- `ConnectionError` — Specifically catches the case where the backend isn't running.

---

**Sidebar: Document List**

```
    st.subheader("Uploaded Documents")
    try:
        docs_response = requests.get(f"{API_URL}/documents/", timeout=10)
        if docs_response.status_code == 200:
            documents = docs_response.json()
            if not documents:
                st.info("No documents uploaded yet.")
            for doc in documents:
                with st.expander(f"■ {doc['filename']}"):
```

**Lines 48-56:** Lists all uploaded documents.
- `st.expander(...)` — Creates a collapsible section for each document. Clicking expands it to show chunk count, page count, and action buttons.
- The document list refreshes on every Streamlit rerun (every interaction), so it's always current.

```
                    if st.button("View Chunks", key=f"chunks_{doc['doc_id']}"):
                        try:
                            chunks_resp = requests.get(
```

```
                                f"{API_URL}/documents/{doc['doc_id']}/chunks", timeout=10
                            )
                            if chunks_resp.status_code == 200:
                                chunks_data = chunks_resp.json()
                                for chunk in chunks_data["chunks"][:3]:
                                    st.text_area(
                                        f"Chunk {chunk['chunk_index']} (Page {chunk['page_number'] + 1},
                                        chunk["text"],
                                        height=100,
                                        disabled=True,
                                        key=f"chunk_{doc['doc_id']}_{chunk['chunk_index']}",
                                    )
                                if len(chunks_data["chunks"]) > 3:
                                    st.caption(f"... and {len(chunks_data['chunks']) - 3} more chunks")
```

**Lines 62-78:** The **Chunk Insights** feature.
- Shows the first 3 chunks as read-only text areas.
- Each chunk displays: its index, page number, character length, and full text.
- `key=f"chunk_{...}"` — Every Streamlit widget needs a unique `key` when created in a loop. Without it, Streamlit can't tell which widget was interacted with.
- Shows "... and N more chunks" if there are more than 3.

```
                    if st.button("■■ Delete", key=f"del_{doc['doc_id']}"):
                        ...
                        st.rerun()
```

**Lines 83-94:** Delete button.
- `st.rerun()` — Forces Streamlit to rerun the entire script, which refreshes the document list (the deleted document disappears).

---

### Main Area: Chat Interface

```
st.title("■ RAG Document Q&A")
st.markdown("Upload documents and ask questions — powered by Claude AI and semantic search")
st.divider()
```

**Lines 102-105:** The main content area header.

```
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
        if message.get("sources"):
            with st.expander("■ Sources"):
                for src in message["sources"]:
                    page_info = f"Page {src['page']}" if src.get("page") else "N/A"
                    st.markdown(f"**{src['filename']}** — {page_info}")
                    st.caption(src["snippet"])
```

**Lines 107-116:** Renders the chat history.
- `st.chat_message(role)` — Creates a chat bubble. `"user"` shows on the right with a person icon, `"assistant"` shows on the left with a bot icon.
- Sources are inside an `st.expander` — visible on click, not cluttering the chat.
- This loop re-renders ALL messages on every rerun, which is why we store them in session state.

```
if prompt := st.chat_input("Ask a question about your documents..."):
```

**Line 119:** The **walrus operator** (`:=`). It assigns the input to `prompt` AND checks if it's truthy, in one line. If the user typed something and pressed Enter, `prompt` contains the text.

```
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)
```

**Lines 121-123:** Immediately shows the user's message in the chat. This gives instant feedback while we wait for the API.

```
    with st.chat_message("assistant"):
        with st.spinner("Searching documents and generating answer..."):
            try:
                response = requests.post(
                    f"{API_URL}/query",
                    json={
                        "question": prompt,
                        "chat_history": st.session_state.chat_history,
                    },
                    timeout=120,
                )
```

**Lines 126-136:** Calls the query API.
- `json={...}` — Sends a JSON body (not form data). `requests` automatically sets `Content-Type: application/json`.
- `chat_history` from session state enables follow-up questions.

```
                if response.status_code == 200:
                    data = response.json()
                    answer = data["answer"]
                    sources = data.get("sources", [])

                    st.markdown(answer)
                    if sources:
                        with st.expander("■ Sources"):
                            ...

                    st.caption(f"■ Answered in {data['time_taken_seconds']}s")
```

**Lines 138-151:** Displays the answer.
- The answer is rendered as Markdown (supports bold, lists, etc.).
- Sources shown in an expandable section.
- Timing displayed as a caption (small, subtle text).

```
                    st.session_state.messages.append({
                        "role": "assistant",
                        "content": answer,
                        "sources": sources,
                    })
                    st.session_state.chat_history.append({"role": "user", "content": prompt})
                    st.session_state.chat_history.append({"role": "assistant", "content": answer})
```

**Lines 154-160:** Updates both state stores.
- `messages` — For rendering the chat UI (includes sources).
- `chat_history` — For sending to the API (plain role/content pairs for context).

## Concepts Covered

| Concept | What It Is |
|---|---|
| **Streamlit** | Python framework for building data/AI web apps with minimal code |

| | |
|---|---|
| **Session State** | Persistent storage across Streamlit reruns |
| **Chat Interface** | `st.chat_message` + `st.chat_input` for conversational UIs |
| **Sidebar Layout** | `with st.sidebar:` for secondary controls |
| **Multipart Upload** | Sending files via HTTP `multipart/form-data` |
| **Walrus Operator** | `:=` for assignment + boolean check in one expression |
| **Widget Keys** | Unique identifiers for Streamlit widgets in loops |
| **Optimistic UI** | Showing the user message immediately, before the API responds |

## Why This Matters for Interviews

"The Streamlit frontend uses `st.chat_message` for a modern chat interface, session state for conversation persistence, and a sidebar for document management. The chat history is sent to the API on every query, enabling follow-up questions. I chose Streamlit over React because for an AI/NLP role, the pipeline matters more than the frontend framework — but I still made the UI polished with expandable sources, chunk inspection, and loading states."

Section 13

# Concepts

# Concepts Glossary

Every concept used in this project, explained in one place.

---

## RAG (Retrieval-Augmented Generation)

A pattern where you first **retrieve** relevant information, then **generate** an answer using that information as context. Instead of trusting the LLM's training data (which can hallucinate), you give it the exact source material to work with.

**Analogy:** Instead of asking someone to answer from memory (hallucination risk), you hand them the textbook opened to the right page and ask them to answer from that.

**In this project:** Documents are uploaded → chunked → embedded → stored. At query time, relevant chunks are retrieved and fed to Claude as context.

---

## Embeddings / Vectors

A **vector** is a list of numbers (e.g., `[0.12, -0.34, 0.56, ...]`) that represents the **meaning** of a piece of text in mathematical space. An **embedding model** converts text into these vectors.

**Key property:** Similar meanings produce similar vectors. "The cat sat on the mat" and "A feline rested on the rug" have vectors close together. "Quantum physics" has a vector far away from both.

**In this project:** We use `all-MiniLM-L6-v2` which produces 384-dimensional vectors. Each dimension captures some aspect of meaning.

---

## Vector Database

A database optimized for storing vectors and finding the **nearest neighbors** — the vectors most similar to a query vector. Regular databases search by exact matches (`WHERE name = 'John'`). Vector databases search by similarity (`find vectors closest to this one`).

**In this project:** ChromaDB stores chunk vectors and supports cosine similarity search.

---

## Cosine Similarity

Measures the **angle** between two vectors. Values range from -1 (opposite) to 1 (identical direction). Preferred over Euclidean distance for text embeddings because it's invariant to vector magnitude — a longer text chunk isn't penalized for having a larger vector.

**Formula:** `cos(θ) = (A · B) / (|A| × |B|)`

---

## HNSW (Hierarchical Navigable Small World)

The algorithm ChromaDB uses internally for fast vector search. Instead of comparing the query against every single vector (slow), HNSW builds a graph structure that allows jumping to the approximate nearest neighbors in O(log n) time.

**Trade-off:** Slightly approximate (might miss the absolute closest vector) but dramatically faster.

---

## Text Chunking

Splitting large documents into smaller pieces. Essential because:
1. **Retrieval precision** — find the specific paragraph, not the whole chapter.
2. **Token limits** — LLMs can only process so much text at once.
3. **Lost in the middle** — LLMs underweight information in the middle of long contexts.

**In this project:** `RecursiveCharacterTextSplitter` splits on paragraph boundaries first, then sentences, then words. 1000-char chunks with 200-char overlap.

---

## Chunk Overlap

Adjacent chunks share characters at their boundaries. If chunk 1 ends with "...the patient was diagnosed with" and chunk 2 starts with "a rare condition called...", the overlap ensures both chunks contain the complete sentence.

**In this project:** 200 characters of overlap. This is ~20% of the chunk size, a common ratio.

---

## BM25 (Best Match 25)

A **keyword-based** ranking algorithm. It scores documents based on:
- **Term Frequency (TF):** How often the search terms appear in a document.
- **Inverse Document Frequency (IDF):** How rare the search terms are across all documents. Rare words (like "pharmaceuticals") score higher than common words (like "the").
- **Document Length:** Normalizes for document length so longer documents aren't unfairly favored.

**In this project:** Used alongside semantic search in hybrid retrieval. BM25 catches exact term matches that semantic search might miss.

---

## Hybrid Search

Combining **semantic search** (meaning-based) with **keyword search** (term-based). Neither is perfect alone:

- Semantic: understands "What are the findings?" matching "The results show..." but might miss exact codes like "ISM001-055".
- Keyword: finds "ISM001-055" perfectly but doesn't understand paraphrases.

Hybrid search uses both and combines their results.

---

## Reciprocal Rank Fusion (RRF)

A method to merge ranked lists from different search systems.

**Formula:** `score(d) = Σ weight / (k + rank(d))`

Where `k` is a constant (typically 60) and `rank(d)` is the item's position in a ranked list.

**Why RRF?** It doesn't need the raw scores from each system — just the rank positions. This avoids the problem of normalizing wildly different score scales (BM25 scores might be 0-15, while cosine distances are 0-1).

**Bonus:** Items that appear in BOTH ranked lists get their scores ADDED, naturally boosting results that both systems agree on.

---

## LangChain

A Python framework for building LLM applications. Provides:
- **Document loaders** (PyPDF, Docx, Text)
- **Text splitters** (RecursiveCharacterTextSplitter)
- **LLM wrappers** (ChatAnthropic for Claude)
- **LCEL (LangChain Expression Language)** for composing chains with `|` pipe syntax

**In this project:** Used for document loading, text splitting, and the prompt → LLM chain.

---

## LCEL (LangChain Expression Language)

A composable syntax for building LLM chains:

```
chain = prompt_template | llm
response = chain.invoke({"question": "..."})
```

The `|` operator pipes the output of one component into the next, like Unix pipes.

---

## Prompt Engineering

Designing the instructions given to an LLM to control its behavior. Key techniques used in this project:
- **Grounding:** "Answer based ONLY on the provided context" — prevents hallucination.
- **Format specification:** "Cite using [Source: filename, Page X]" — ensures consistent, parseable output.
- **Honesty instruction:** "If context doesn't contain the answer, say so" — prevents fabrication.

---

## FastAPI

A modern Python web framework for building APIs. Key features used:
- **Automatic validation** via Pydantic models.
- **Auto-generated docs** at `/docs` (Swagger UI).
- **Async support** for file uploads.
- **Routers** for organizing endpoints.

---

## Pydantic

A data validation library. Define a class with type annotations, and Pydantic:
- Validates incoming data matches the types.
- Converts compatible types (string "123" → int 123).
- Generates JSON schemas (used by FastAPI for Swagger docs).
- Serializes to JSON automatically.

---

## CORS (Cross-Origin Resource Sharing)

A browser security mechanism that blocks requests between different origins (protocol + domain + port). `localhost:8501` (Streamlit) → `localhost:8000` (API) is a cross-origin request. CORS middleware tells the browser "this is allowed."

---

## Streamlit Session State

Streamlit reruns the entire script on every user interaction. `st.session_state` is a dictionary that persists across reruns, used to maintain chat history and message display.

---

## Singleton Pattern / Lazy Loading

Creating a single shared instance of an expensive resource (embedding model, DB connection) on first use, then reusing it. Avoids:
- Loading the model multiple times (wasteful).
- Loading at import time (slows startup, breaks tests).

```
_model = None
def get_model():
    global _model
    if _model is None:
        _model = SentenceTransformer(...)  # Only runs once
    return _model
```

---

## UUID (Universally Unique Identifier)

A 128-bit identifier that's virtually guaranteed unique. We use UUID v4 (random) for document IDs. With 122 bits of randomness, the probability of a collision is astronomically low — you'd need to generate ~2.7 quintillion UUIDs to have a 50% chance of one duplicate.

Section 14

# Interview Prep

# Interview Prep — Questions & Talking Points

Questions you should be ready to answer about this project, organized by topic.

---

## RAG Architecture

**Q: What is RAG and why did you use it?**

RAG stands for Retrieval-Augmented Generation. Instead of relying on the LLM's training data (which can be outdated or hallucinated), I retrieve relevant passages from actual uploaded documents and provide them as context. This grounds the answer in real sources and enables citation. It's the dominant pattern in enterprise AI applications because it combines the LLM's reasoning ability with trustworthy source data.

**Q: Why not just put the entire document into Claude's context window?**

Three reasons:
1. **Cost** — Claude charges per token. Sending a 50-page PDF on every query is expensive.
2. **Precision** — The "lost in the middle" problem: research shows LLMs pay less attention to information in the middle of long contexts. By retrieving only the 5 most relevant chunks, the LLM focuses on what matters.
3. **Scale** — Context windows have limits. If someone uploads 20 documents, they won't fit. RAG scales to any number of documents.

**Q: Walk me through what happens when a user asks a question.**

1. The question is sent to the FastAPI backend.
2. The question is embedded into a 384-dimensional vector using sentence-transformers.
3. Two parallel searches run: semantic search (ChromaDB cosine similarity) and keyword search (BM25).
4. Results are merged using Reciprocal Rank Fusion, keeping the top 5 chunks.
5. Those chunks are formatted with source labels and sent to Claude with a citation-enforcing prompt.
6. Claude generates an answer citing specific files and pages.
7. The response includes the answer, structured source references, and timing.

---

## Embeddings

**Q: What embedding model did you use and why?**

`all-MiniLM-L6-v2` from sentence-transformers. It produces 384-dimensional vectors, runs locally with no API cost, and is the most popular lightweight embedding model (90M+ downloads). I chose it over OpenAI's embeddings to demonstrate that I understand the embedding layer itself — not just API calls — and to avoid requiring a second API key.

**Q: What does a 384-dimensional vector mean?**

Each dimension captures some learned aspect of the text's meaning. We can't interpret individual dimensions, but the model learned (through training on millions of sentence pairs) that similar texts should have similar vectors. The 384 dimensions are enough to encode nuanced meaning differences while keeping computation fast.

**Q: How do you compare two vectors?**

Cosine similarity — it measures the angle between vectors. A value of 1.0 means identical direction (same meaning), 0 means unrelated, -1 means opposite. I chose cosine over Euclidean distance because cosine is invariant to vector magnitude, so longer chunks aren't penalized.

---

## Chunking

**Q: Why chunk size 1000 with overlap 200?**

Chunk size 1000 (~150-200 words) is the sweet spot for RAG. Smaller chunks (200 chars) lose context — a chunk might just say "it increased by 40%" without saying what "it" refers to. Larger chunks (5000 chars) dilute retrieval precision — you'd retrieve an entire section when only one sentence is relevant.

The 200-char overlap (20% of chunk size) prevents information loss at boundaries. If an important sentence is split between chunks, the overlap ensures both adjacent chunks contain it.

**Q: Why RecursiveCharacterTextSplitter specifically?**

It tries to split at natural boundaries in this order: paragraph breaks → line breaks → spaces → characters. So a 1000-char chunk is more likely to end at a paragraph boundary than in the middle of a word. Other splitters (like fixed-size or sentence-based) either cut awkwardly or produce uneven chunks.

---

## Hybrid Search

**Q: Why not just use semantic search?**

Semantic search fails on exact terms. If a document mentions the drug code "ISM001-055" and someone asks about it, semantic search might not find it because "ISM001-055" doesn't have a clear semantic meaning — it's just a code. BM25 keyword search finds it instantly because it's an exact token match.

Conversely, keyword search fails on paraphrases. "What were the outcomes?" won't match a chunk that says "The results indicate..." because the words are different. Semantic search understands they mean the same thing.

Hybrid search combines both, getting the best of each.

**Q: Explain Reciprocal Rank Fusion.**

RRF merges ranked lists from different search systems using only rank positions, not raw scores. The formula is `score = weight / (k + rank + 1)` where k=60 is a constant.

The beauty is that items appearing in BOTH lists get their scores added. If a chunk is ranked #1 in semantic search AND #2 in BM25, it gets a higher combined score than a chunk that's #1 in one but absent from the other. This naturally rewards consensus between the two systems.

I weighted it 70/30 favoring semantic search because semantic generally performs better for natural language questions. BM25 is the safety net for exact-match cases.

**Q: Why rebuild the BM25 index on every query? Isn't that slow?**

For a portfolio project with a few documents, it's fast enough (milliseconds). In production with millions of chunks, I'd use a persistent BM25 index like Elasticsearch or a dedicated keyword search service. The trade-off here is simplicity — no extra infrastructure to manage.

---

## Prompt Engineering

**Q: How did you prevent hallucination?**

The prompt says "Answer based ONLY on the provided context excerpts." This grounds Claude's answer in the retrieved chunks. I also instruct it to explicitly say "the documents don't contain this information" rather than guessing. And I set `temperature=0` for deterministic, conservative output.

**Q: Why did you include the citation format in the prompt?**

By specifying the exact format `[Source: filename, Page X]` and labeling each chunk with the same format in the context, Claude can directly copy the citation. This is more reliable than asking Claude to "cite your sources" without a format — ambiguity leads to inconsistent output.

---

## Architecture & Design

**Q: Why separate the frontend and backend?**

This mirrors production architecture. The API can be consumed by multiple clients (Streamlit, mobile app, CLI tool). They can be deployed, scaled, and updated independently. It also makes testing easier — I can test the API without the UI.

**Q: Why FastAPI over Flask or Django?**

FastAPI gives me automatic request validation (via Pydantic), auto-generated Swagger docs, async support for file uploads, and type safety — all with less boilerplate than Django and more features than Flask.

**Q: What's the thin controller pattern you used?**

Routers only handle HTTP concerns (validation, error codes, timing). All business logic lives in the service layer (document_processor, retriever, qa_chain). This means I can test the RAG pipeline directly by calling `qa_chain.answer_question()` without HTTP, and I could add a CLI interface without duplicating logic.

---

## Tools & Libraries

**Q: Why ChromaDB over FAISS/Pinecone?**

- **vs FAISS:** FAISS is a raw index without metadata support. I need metadata (filename, page number) for citations. ChromaDB provides metadata filtering, persistence, and a clean Python API.
- **vs Pinecone:** Pinecone requires an account, has vendor lock-in, and costs money at scale. ChromaDB runs locally and is free.

**Q: Why sentence-transformers over OpenAI embeddings?**

No extra API cost, runs offline for faster development, and demonstrates I understand the model layer rather than just API calls. In production, I'd benchmark both — OpenAI's ada-002 might be better for specific domains.

**Q: How does Docker Compose help here?**

It orchestrates two services (API on port 8000, Streamlit on port 8501) with one command. The services can communicate via Docker's internal network (`http://api:8000` instead of localhost). This is the standard way to run multi-service applications.

---

## Growth & Production Readiness

**Q: What would you change for a production deployment?**

1. **Persistent BM25 index** — Use Elasticsearch instead of rebuilding per query.
2. **Async processing** — Upload processing on a background task queue (Celery) so the API doesn't block.
3. **Authentication** — API keys or OAuth for access control.
4. **Rate limiting** — Prevent abuse of the Claude API.

5. **Persistent storage** — S3 for uploaded files, a managed vector DB like Pinecone or Weaviate.
6. **Evaluation** — Automated RAG evaluation with metrics like faithfulness, relevance, and answer correctness.
7. **Restrict CORS** — Replace `allow_origins=["*"]` with the actual frontend domain.

**Q: What would you do differently if you built this again?**

I'd add a **reranker** between retrieval and generation. After hybrid search returns the top 5 chunks, a cross-encoder model (like `ms-marco-MiniLM`) would rescore them based on the actual question-chunk pair. This typically improves answer quality by 10-20% because cross-encoders consider the full interaction between question and passage, not just vector similarity.